

Securing An Internship In The AI Era

From 1337 To The Workplace

Jalal Maaouni

May 8, 2026

Plan

- 1 Opening Message
- 2 What AI Changed
- 3 The New Intern Profile
- 4 Using AI Properly

- 5 Building Proof
- 6 Finding Opportunities
- 7 Interview Preparation
- 8 Common Mistakes

OPENING MESSAGE

The uncomfortable truth

Before

A student could stand out by saying: “I know how to code.”

Now

That sentence is weak. AI can produce code-like output, clean CV text, polished README files, and generic interview answers.

In the AI era, internships are not won by claiming competence. They are won by proving competence.

**The market did not kill junior developers.
It killed tolerance for shallow junior developers.**

A student who only asks AI to generate code is replaceable. A student who understands fundamentals, uses AI critically, builds real things, and communicates clearly is more valuable than before.

What Oracle taught me

- ▶ Big companies value **reliability** more than cleverness.
- ▶ Code usually lives inside large systems, not isolated exercises.
- ▶ Asking good questions is a professional skill.
- ▶ Understanding existing code is often harder than writing new code.
- ▶ Documentation, tests, reviews, and communication are not optional extras.

The useful intern is not the one who pretends to know everything. The useful intern reduces uncertainty for the team.

WHAT AI CHANGED

The market changed because the baseline changed

AI compressed simple work

- ▶ boilerplate code;
- ▶ simple CRUD endpoints;
- ▶ first draft documentation;
- ▶ generic scripts;
- ▶ basic debugging suggestions.

Recruiters became more skeptical

- ▶ polished CVs are cheaper;
- ▶ cover letters are less meaningful;
- ▶ shallow projects are easier to fake;
- ▶ take-home work is harder to trust.

The filter moved from presentation to evidence.

Before AI vs. after AI

Before AI	After AI
A clean CV helped you stand out.	A clean CV is expected and easy to generate.
Basic projects showed initiative.	Basic projects may look AI-generated unless explained deeply.
Take-home tasks proved skill.	Take-home tasks are less trusted without discussion and defense.
Knowing syntax mattered more.	Understanding, debugging, testing, and judgment matter more.

The new question is not: “Can you produce code?” It is: “Can you understand, verify, improve, and own code?”

THE NEW INTERN PROFILE

What companies want from interns now

Area	What it means
Fundamentals	Data structures, algorithms, Git, databases, APIs, debugging, testing.
Project proof	Repositories, demos, deployment, readable README files, real tradeoffs.
AI fluency	Can use AI, but can also verify, correct, and explain the result.
Communication	Writes clearly, asks precise questions, reports progress honestly.
Ownership	Can take a task from ambiguity to a working result.

The best internship candidate is not the most advanced student. It is the student a team can trust with real work.

The skill stack

Hard skills

- ▶ programming fundamentals;
- ▶ algorithms and complexity;
- ▶ Linux and Git;
- ▶ databases and APIs;
- ▶ testing and debugging;
- ▶ deployment basics.

Professional skills

- ▶ explaining decisions;
- ▶ reading existing code;
- ▶ asking focused questions;
- ▶ writing useful documentation;
- ▶ receiving criticism;
- ▶ finishing small tasks cleanly.

Technical skill gets you considered. Professional maturity gets you trusted.

USING AI PROPERLY

Bad AI usage

- ▶ “Write my CV so I sound senior.”
- ▶ “Generate a project and I will push it to GitHub.”
- ▶ “Solve this take-home test for me.”
- ▶ “Explain this code five minutes before the interview.”
- ▶ “Make my experience sound bigger than it is.”

This creates a dangerous gap: the application looks strong, but the candidate cannot defend it.

Good AI usage

- ▶ “Review my CV against this internship offer.”
- ▶ “Ask me hard questions about this project.”
- ▶ “Find edge cases in this function.”
- ▶ “Explain this unfamiliar codebase module by module.”
- ▶ “Help me write tests for this API.”
- ▶ “Compare these two design choices and challenge my assumptions.”

Use AI as a reviewer, tutor, debugger, and sparring partner. Do not use it as a ghostwriter.

A better AI prompt

Weak prompt

```
1 Write me a good CV for a software internship.
```

Stronger prompt

```
1 Act as a strict technical recruiter.  
2 Review my CV for a backend internship.  
3 Find weak claims, missing proof, vague wording,  
4 and questions I should be ready to answer.  
5 Do not rewrite everything. Criticize first.
```

A good AI prompt should make you more accountable, not less accountable.

BUILDING PROOF

The proof pyramid

Level	Proof
Weak	"I am passionate and motivated."
Better	"I know Python, C, Git, Linux, and SQL."
Stronger	"Here is a repository with tests, README, and clean history."
Strongest	"Here is a deployed project, the design tradeoffs, the bugs I fixed, and what I would improve next."

Your application should make verification easy.

A credible GitHub repository

Must have

- ▶ clear README;
- ▶ installation steps;
- ▶ screenshots or demo link;
- ▶ tests or examples;
- ▶ meaningful commit history.

Strong signal

- ▶ architecture explanation;
- ▶ tradeoff discussion;
- ▶ known limitations;
- ▶ issue list or roadmap;
- ▶ deployment instructions.

A recruiter may scan your project quickly. A technical interviewer may interrogate it deeply. Prepare for both.

CV: what to remove

- ▶ Empty phrases: “dynamic”, “passionate”, “hard worker”, “problem solver”.
- ▶ Skill lists with no evidence.
- ▶ Projects with no links, no context, and no measurable output.
- ▶ Inflated titles or claims that collapse under questioning.
- ▶ Long paragraphs that hide the important facts.

A CV is not a biography. It is an index of evidence.

CV: what to add

- ▶ One-page structure.
- ▶ Projects described with action, technology, and result.
- ▶ Links to GitHub, demo, or technical write-up.
- ▶ Clear technical stack, but only skills you can defend.
- ▶ Internship-relevant 1337 projects.
- ▶ Short bullets showing impact and constraints.

Example 5.8. Better bullet

Built a FastAPI + PostgreSQL appointment system with role-based access, Docker deployment, and integration tests for core patient workflows.

FINDING OPPORTUNITIES

Do not only apply through portals

Low signal

- ▶ upload CV;
- ▶ paste generic cover letter;
- ▶ wait;
- ▶ complain about no answer.

Higher signal

- ▶ target specific teams;
- ▶ show one relevant project;
- ▶ contact alumni and engineers;
- ▶ ask precise questions;
- ▶ follow up respectfully.

The goal is not to apply everywhere. The goal is to become easy to recommend.

A concise outreach message

Example

```
1 Hello [Name],  
2  
3 I am a 1337 student interested in backend engineering.  
4 I saw that your team works on [specific area].  
5  
6 I recently built [project] using [stack], with tests and Docker deployment:  
7 [link]  
8  
9 If your team accepts interns, I would appreciate any advice  
10 on how to apply properly or what skills to strengthen.  
11  
12 Thank you,  
13 [Name]
```

Specific, short, respectful, and backed by proof. That is enough.

INTERVIEW PREPARATION

Four interview types

Type	What to prepare
Algorithmic	Data structures, algorithms, complexity.
Project-based	Architecture, bugs, tradeoffs, personal contribution, next improvements.
Behavioral	Failure, teamwork, feedback, conflict, deadline pressure, learning fast.
AI-assisted	How you use AI, verify output, catch bugs, and explain final decisions.

The project interview is where fake competence collapses fastest.

How to defend a project

For each serious project, prepare answers to these questions:

- ▶ What problem does it solve?
- ▶ Why did you choose this architecture?
- ▶ What was the hardest bug?
- ▶ What would break at 10x more users or data?
- ▶ What did you test?
- ▶ What would you improve if you had one more month?
- ▶ Which parts were helped by AI, and how did you verify them?

If you cannot explain a project deeply, it is not yet an asset. It is a liability.

Behavioral stories

Prepare real stories before the interview.

Useful story types

- ▶ a hard bug;
- ▶ a failed project;
- ▶ a conflict in a team;
- ▶ learning a tool quickly;
- ▶ receiving harsh feedback.

What interviewers look for

- ▶ honesty;
- ▶ self-awareness;
- ▶ responsibility;
- ▶ learning;
- ▶ clear communication.

Do not present yourself as flawless. Present yourself as someone who improves.

COMMON MISTAKES

Mistakes that kill internship chances

- ▶ Applying only at the last minute.
- ▶ Using the same CV for every role.
- ▶ Listing technologies you cannot explain.
- ▶ Having GitHub projects that do not run.
- ▶ No README, no tests, no demo, no screenshots.
- ▶ Weak English or French communication in professional messages.
- ▶ Waiting for school projects to be enough.

Most students do not lose because they are incapable. They lose because their proof is weak, late, or badly presented.

The 1337 advantage

What 1337 gives you

- ▶ peer learning;
- ▶ autonomy;
- ▶ pressure;
- ▶ debugging stamina;
- ▶ low-level foundations;
- ▶ project culture.

What you must add

- ▶ public proof;
- ▶ professional writing;
- ▶ deployment;
- ▶ testing;
- ▶ product thinking;
- ▶ interview discipline.

1337 can make you technically tough. You still need to make yourself professionally legible.

ACTION PLAN

A 4-week internship sprint

Week	Focus
1	Fix CV, LinkedIn, GitHub profile, and choose target roles.
2	Polish one flagship project: README, tests, screenshots, deployment, roadmap.
3	Apply seriously: targeted applications, alumni outreach, recruiter messages, follow-ups.
4	Mock interviews: algorithms, project defense, behavioral stories, AI usage explanation.

Do not wait until you need an internship to start becoming visible.

Weekly rhythm

- ▶ **Build:** improve one project feature or fix one real issue.
- ▶ **Write:** document one technical lesson learned.
- ▶ **Apply:** send targeted applications and messages.
- ▶ **Practice:** solve interview problems and explain them aloud.
- ▶ **Review:** ask AI, peers, or mentors to criticize your work.

Consistency beats panic. The market rewards visible accumulation of proof.

Final checklist

Before applying, ask:

- ▶ Can I defend every skill on my CV?
- ▶ Does my main project run from the README instructions?
- ▶ Do I have at least one project with tests or clear examples?
- ▶ Can I explain my architecture choices?
- ▶ Can I discuss where I used AI and how I verified the output?
- ▶ Do I have a short, specific outreach message?
- ▶ Have I practiced explaining my work out loud?

If the answer is no, fix the proof before sending more applications.

CLOSING

**AI made weak applications easier to produce.
It also made real competence more valuable.**

Your job is to become the student whose work can be checked, whose reasoning can be trusted, and whose progress is visible before the interview starts.

One sentence to remember

Do not ask: “How do I get an internship?”

Ask: “What proof would make someone confident recommending me?”

Build that proof deliberately, then the internship search becomes much less random.